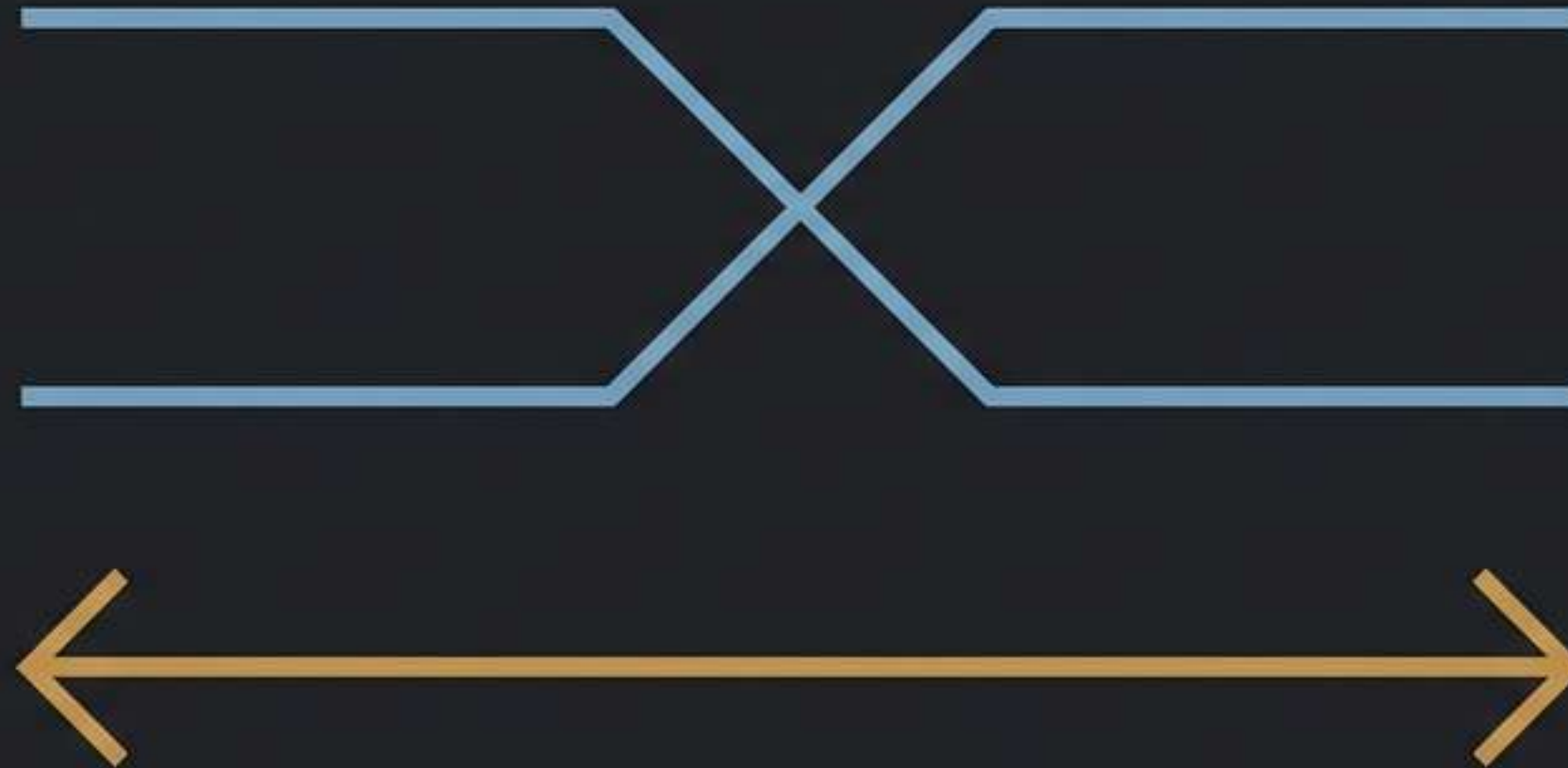


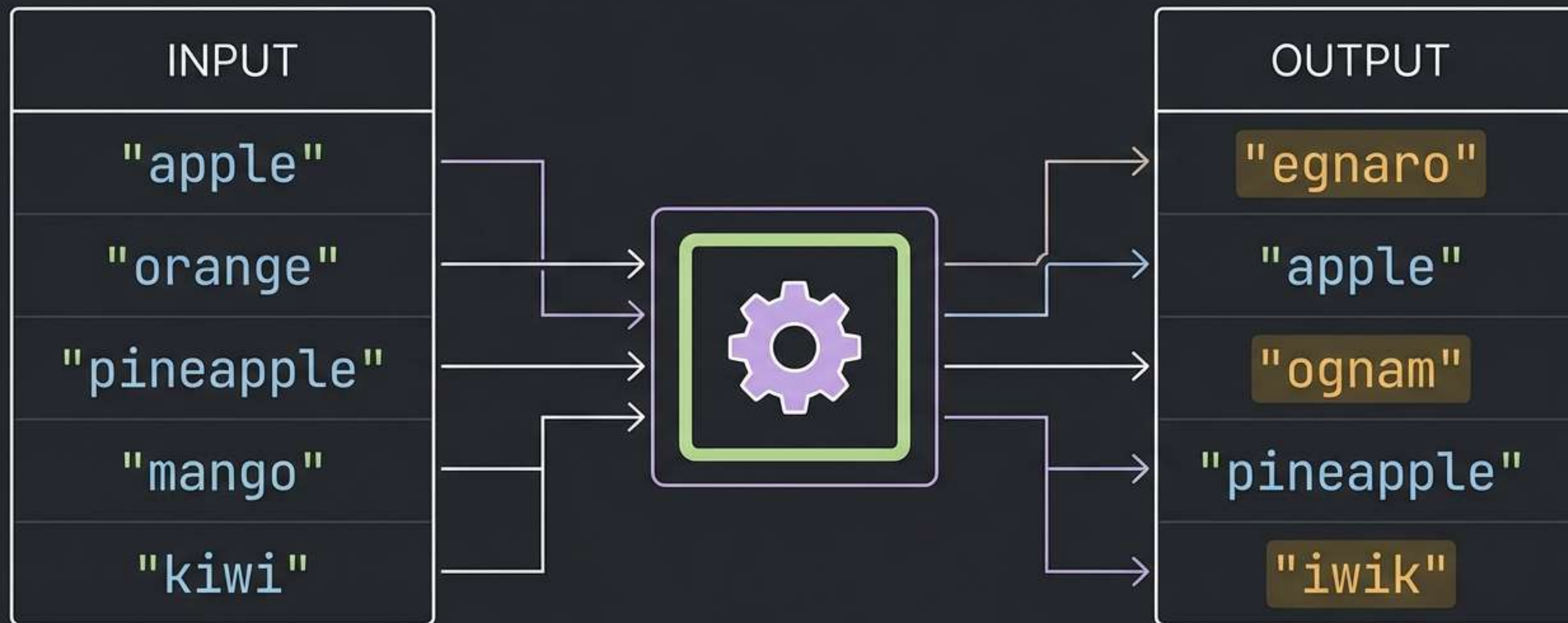
Algorithmic Line Permutation & Reversal Logic

2024 Sep OPPE 1 – Set 3 | Problem Analysis & Solution



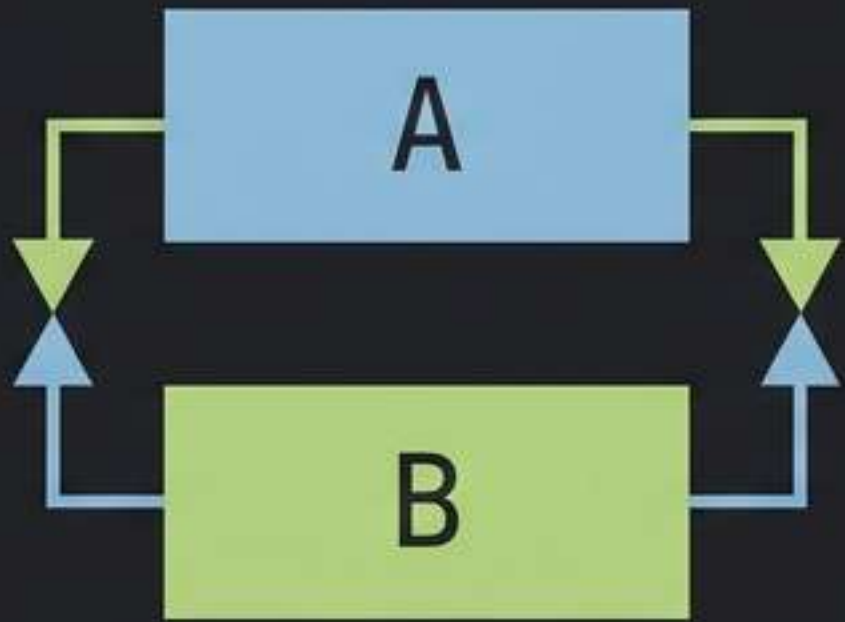
Language: Python 3.10+

The Transformation Challenge



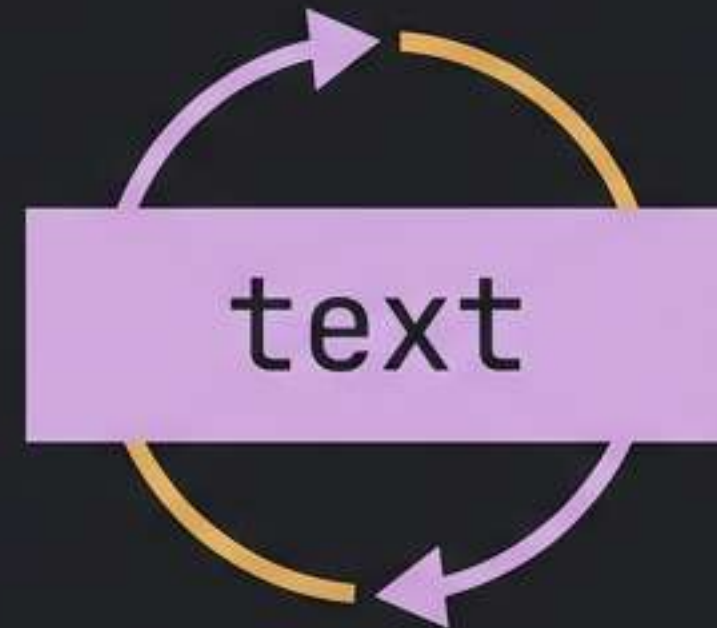
Two Rules of Engagement

Rule A: Structure



Swap every two consecutive lines.

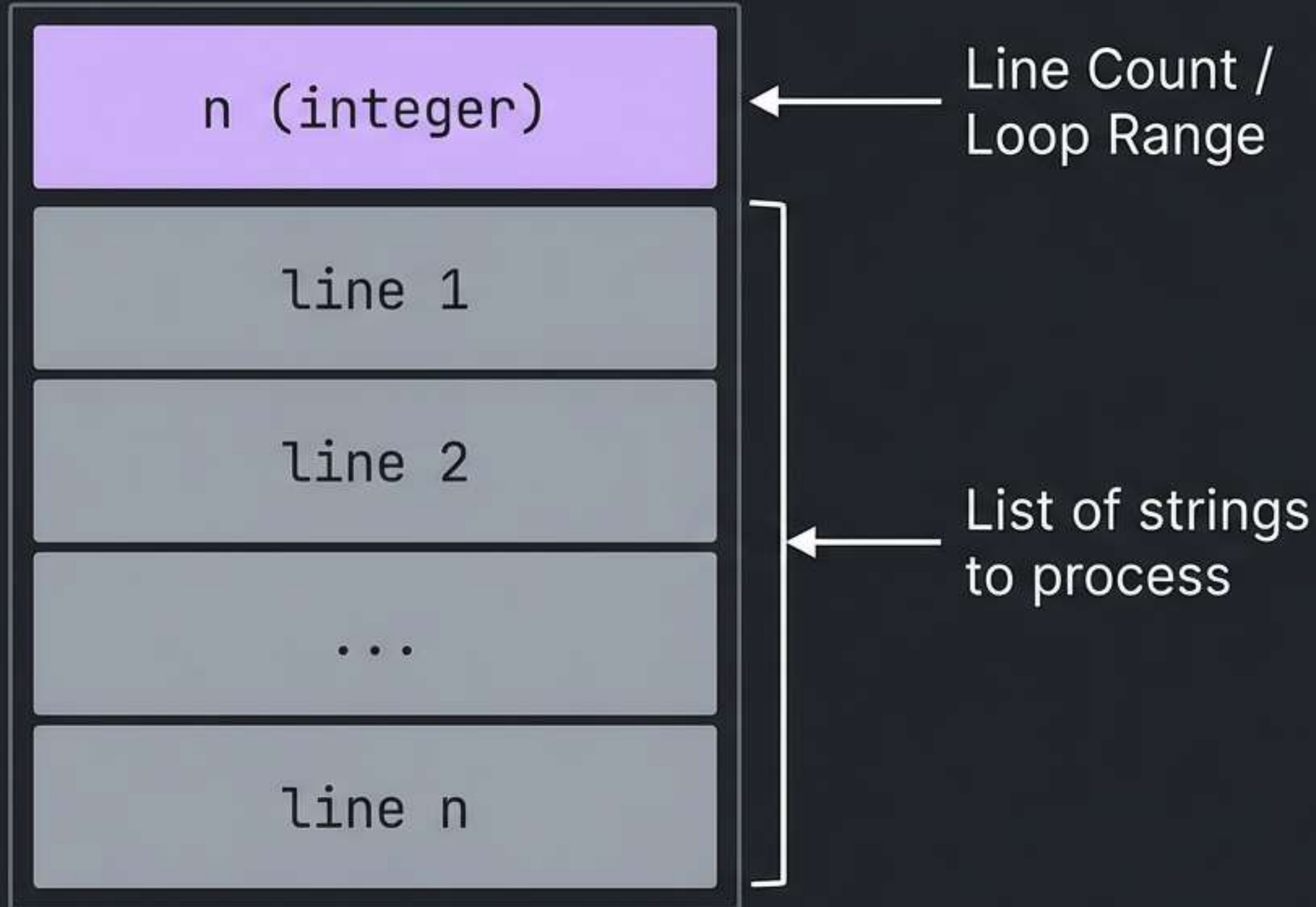
Rule B: Content



Reverse every odd-numbered line in the final sequence.

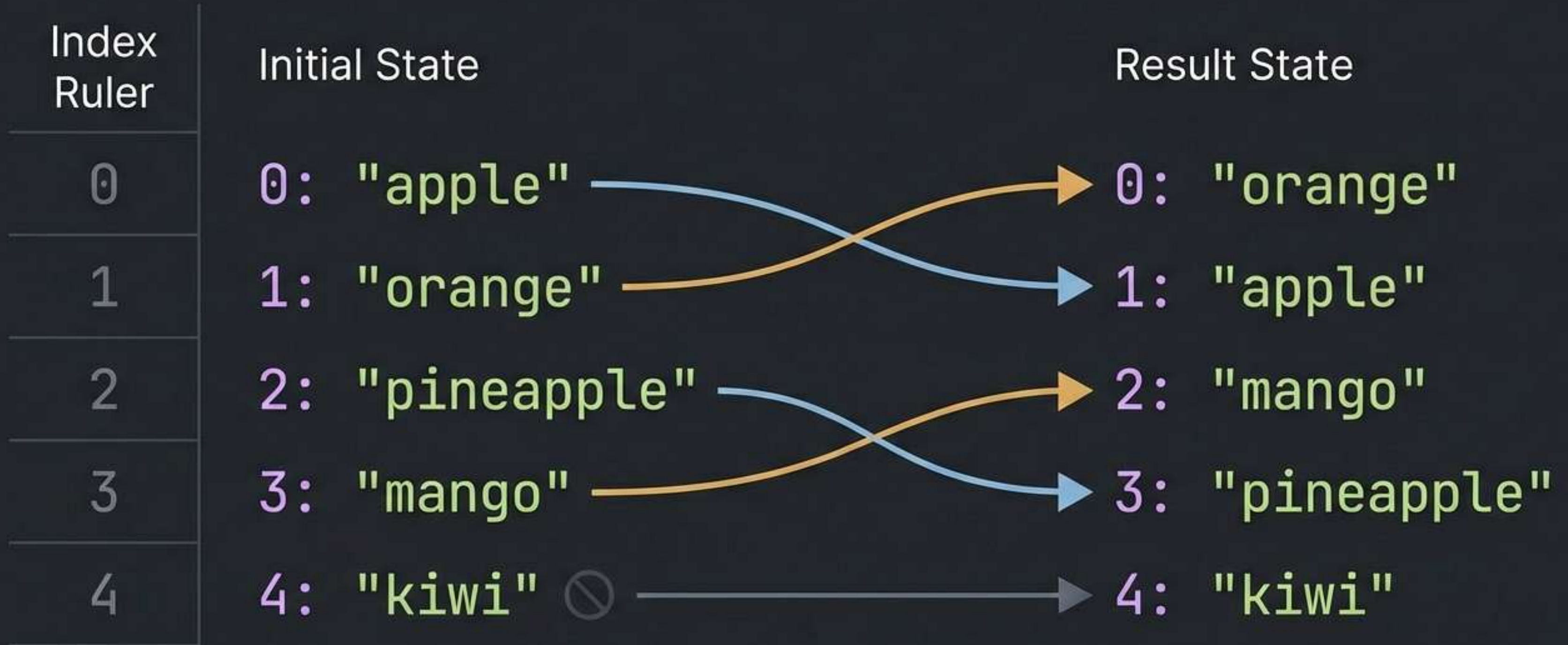
SYSTEM NOTE: This is an I/O problem. The solution requires full code implementation for input ingestion and output printing.

Input Specification & Data Structure



1. The first line defines the integer `n`.
2. The subsequent `n` lines form the list of strings.
3. Code must handle integer parsing separately from string storage.

Execution Step 1: The Pair Swap



Execution Step 2: The Content Reversal

0: "orange"	→ "egnaro"	Odd Line 1 / Index 0
1: "apple"		
2: "mango"	→ "ognam"	Odd Line 3 / Index 2
3: "pineapple"		
4: "kiwi"	→ "iwik"	Odd Line 5 / Index 4

Transformation applied only to odd-numbered lines (human counting).

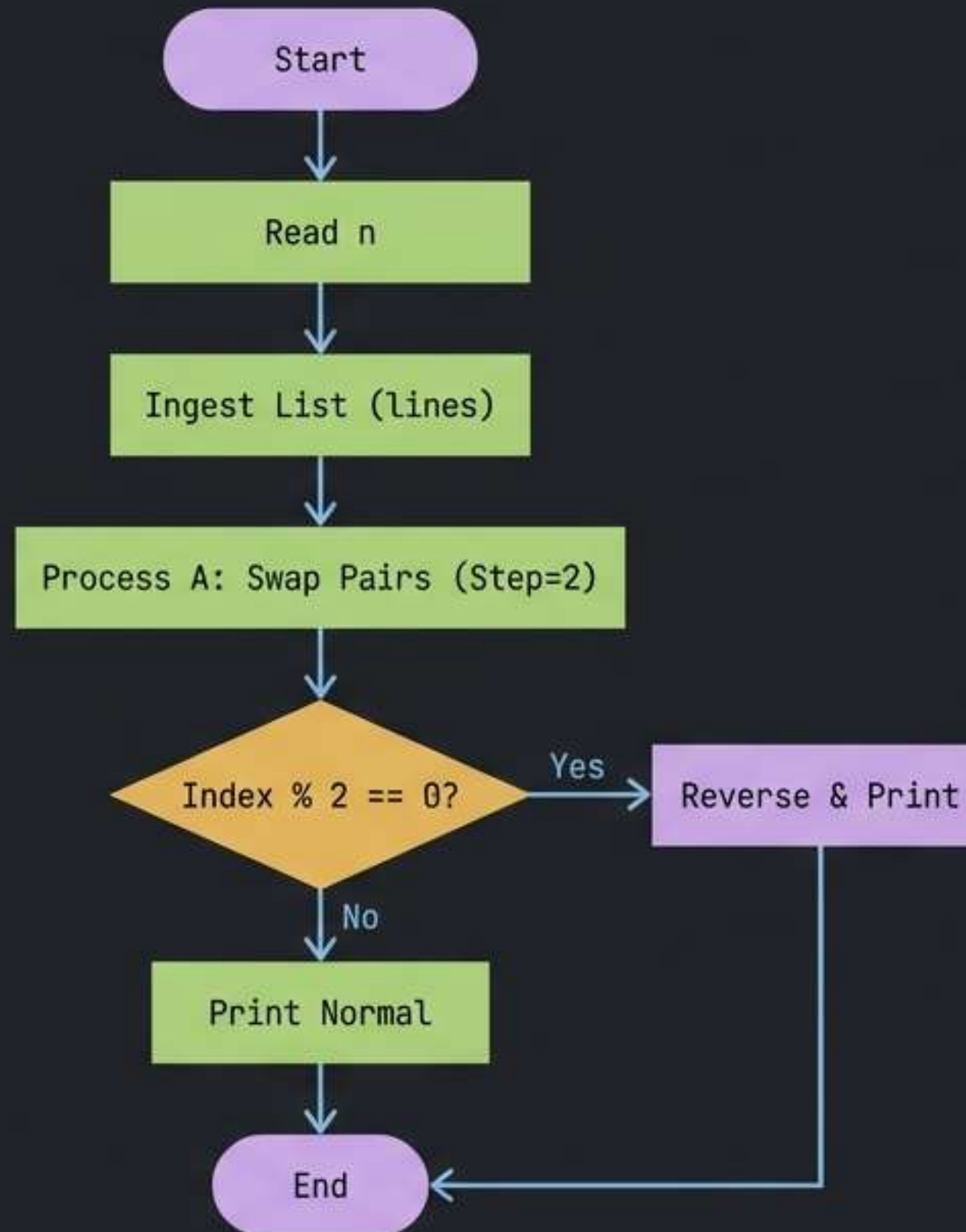
The Logic Trap: Indexing Alignment

Human vs. Machine		
Line 1	(Odd)	Index 0 (Even)
Line 3	(Odd)	Index 2 (Even)
Line 5	(Odd)	Index 4 (Even)

Target Odd Lines = Target Even Indices

`if index % 2 == 0: Reverse String`

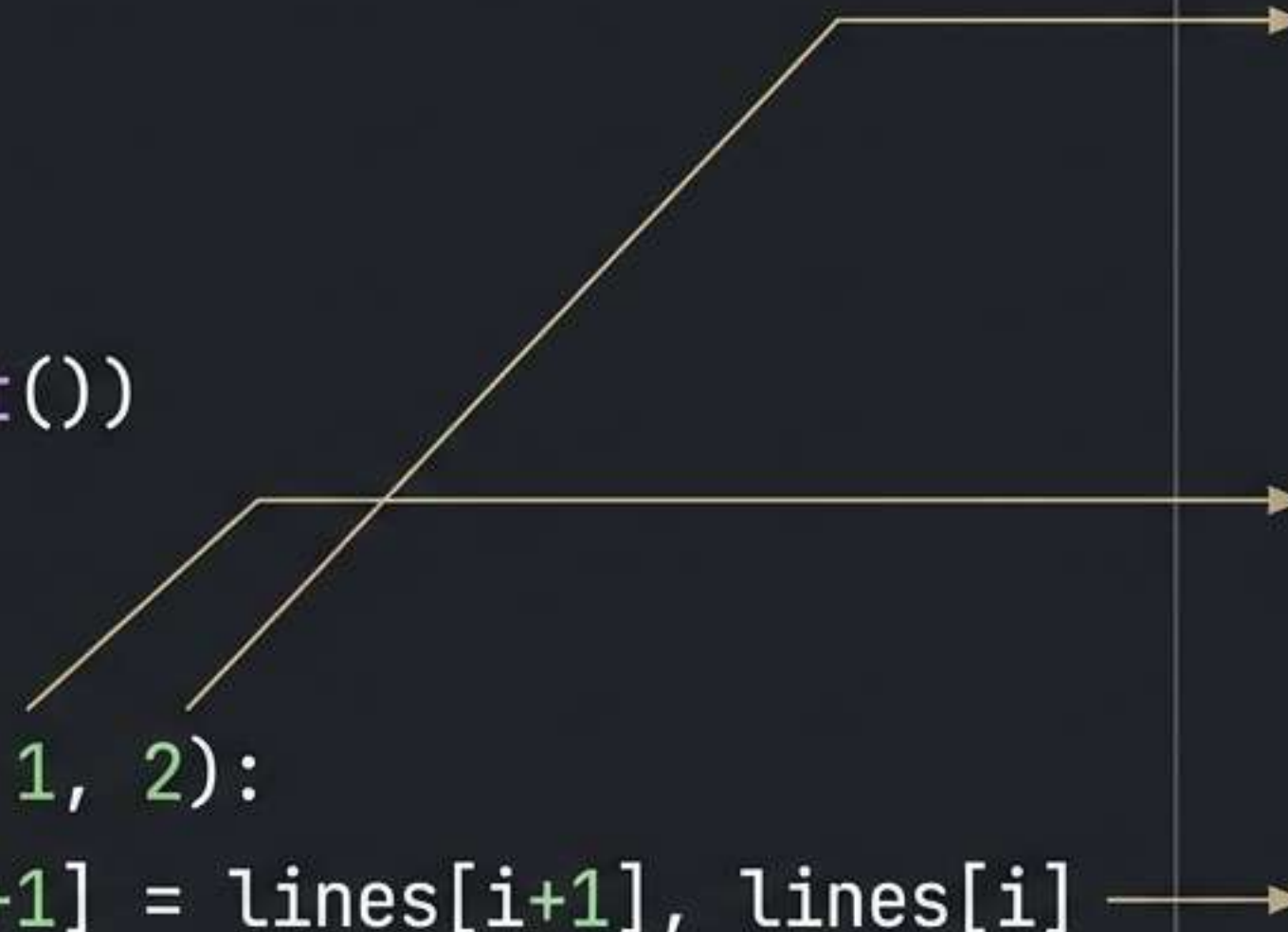
Solution Architecture



Implementation: Ingestion & Swapping

```
n = int(input())
lines = []
for _ in range(n):
    lines.append(input())

# Swap Logic
for i in range(0, n - 1, 2):
    lines[i], lines[i+1] = lines[i+1], lines[i]
```



Step Size 2:

Jumps directly to the start of each pair.

Stop Condition:

Ensures we don't go out of bounds if n is odd.

Tuple Unpacking:

Pythonic simultaneous swap.

Implementation: Conditional Output

```
# Print with reversal on odd-numbered lines  
(Even Index)
```

```
for i in range(n):
```

```
    if i % 2 == 0:
```

```
        print(lines[i][::-1])
```

```
    else:
```

```
        print(lines[i])
```

Logic Mapping

Even Index = Odd
Line Number.

Slicing

[start : stop : step]. A
step of -1 reverses
the string instantly.

The Full Executable Solution

```
n = int(input())
lines = []

# 1. Ingestion
for _ in range(n):
    lines.append(input())

# 2. Structure Swap
for i in range(0, n - 1, 2):
    lines[i], lines[i + 1] = lines[i + 1], lines[i]

# 3. Process & Output
for i in range(n):
    if i % 2 == 0:
        print(lines[i][::-1])
    else:
        print(lines[i])
```

[✓] Read Input

[✓] Store List

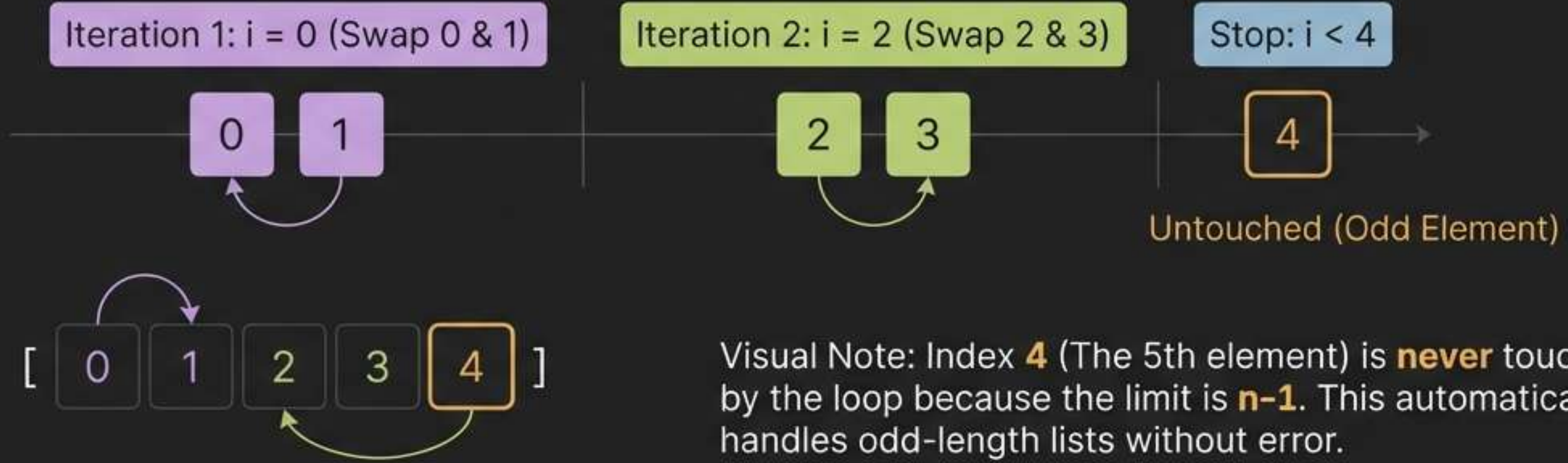
[✓] Swap Pairs

[✓] Reverse Even Indices

Edge Case Analysis: The Odd Element

`range(0, n - 1, 2)`

Example Scenario: Input $n = 5$



Complexity & Performance

Time Complexity

$O(n)$

Linear. We iterate through the list exactly 3 times (Read, Swap, Print).

Space Complexity

$O(n)$

Linear. We must store all ' n ' lines in memory to perform the swaps.

Key Takeaways

- 1 Step Size:** Use `range(start, stop, step)` to safely process items in pairs.
- 2 Indexing Trap:** Always map 'Odd Positions' to 'Even Indices' (0-based).
- 3 Slicing Power:** Master `[::-1]` for efficient string reversal.
- 4 Order of Operations:** Swap the structure before modifying the content.